hashlock.

Security Audit

Coinsub (DeFi)

March 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

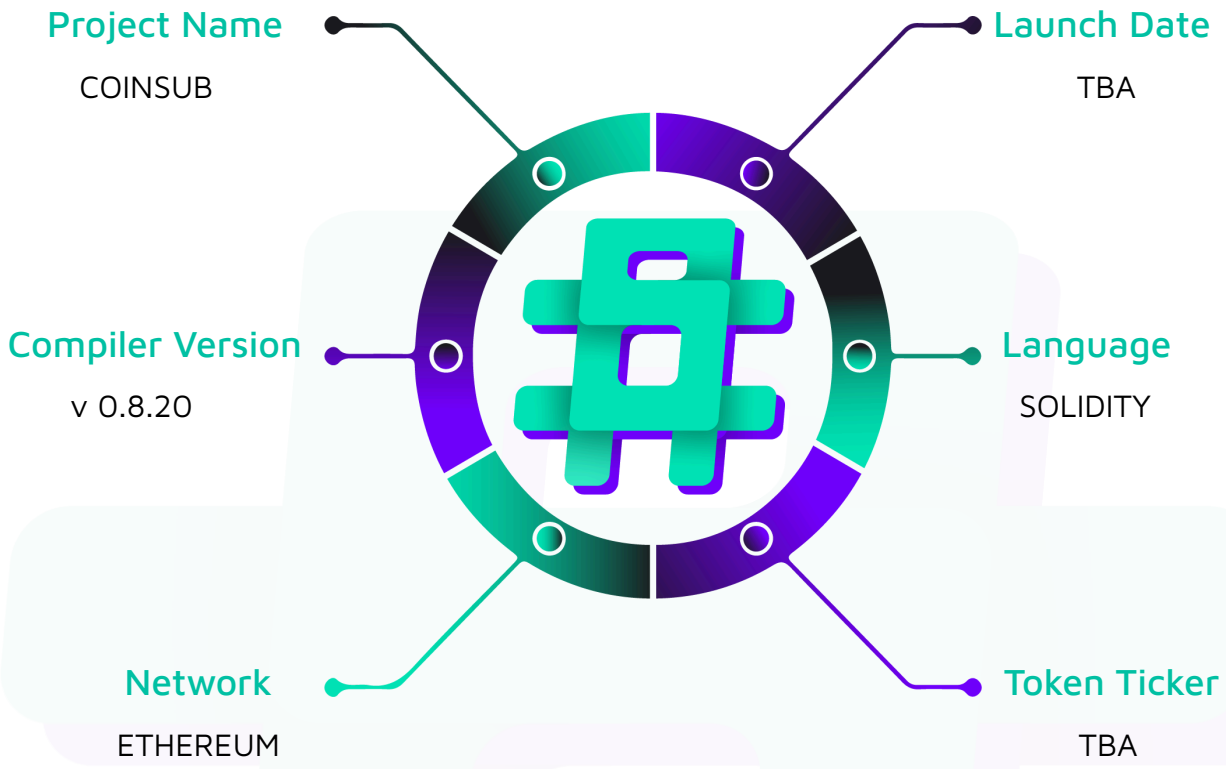
The Coinsub team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Coinsub offers the most advanced Crypto Payment Orchestration Platform (CPOP), enabling businesses to seamlessly integrate crypto payments without the burden of infrastructure development. As part of our commitment to security, scalability, and efficiency, we are releasing a new smart contract update designed to enhance our API capabilities. This update introduces improved transaction batching, advanced recurring payment logic, optimized reliability for processing large transaction volumes, and expanded multi-chain support, ensuring seamless interoperability for payment processors, ISOs, and merchant networks. Backed by a comprehensive audit, this update reinforces Coinsub's position as the most secure and reliable crypto payment infrastructure, ready to support the next generation of global transactions.

Project Name: Coinsub
Project Type: Defi
Compiler Version: 0.8.20
Website: www.coinsub.io
Logo:

Visualised Context:



Project Visuals:

Audit Scope

We at Hashlock audited the solidity code within the Coinsub project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Coinsub Smart Contracts
Platform	**EVM / Solidity**
Audit Date	**March, 2025**
Contract 1	AddressRegistry.sol
Contract 1 MD5 Hash	23dff27ee0fce9873529e387e5ca8d1d
Contract 2	AuthorizationManager.sol
Contract 2 MD5 Hash	432ec9011d0f69f143c548fdafc7da7f
Contract 3	NumberRegistry.sol
Contract 3 MD5 Hash	90d008c3148ca83fc96ad9d33acf4807
Contract 4	PaymentAgreement.sol
Contract 4 MD5 Hash	afcb1822db3b61791d91d3014ca3592e
Contract 5	RecurringPaymentAgreement.sol
Contract 5 MD5 Hash	748f189d9dde64a37977fccc3742f8ff
Contract 6	TokenRegistry.sol
Contract 6 MD5 Hash	daefe76b07de6348fe6b36c65af6341f
GitHub Commit Hash	2b000f784928c517095b70b64e2721e12b1f3b5a

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

We initially identified some vulnerabilities that have since been addressed.

Hashlock found:

1 Medium severity vulnerability

2 Low severity vulnerabilities

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
AddressRegistry.sol The AddressRegistry maps identities, businesses, or other recognized entities to their on-chain addresses. **Functionality Confirmed:** - Prevents malicious actors from falsely claiming names. - Ensures consistency across networks. - Only authorized entities can modify the registry **Key Features:** - Maps verified entities to addresses. - Facilitates human-readable identifiers in cryptographic agreements. - Ensures correct address resolution for payments.	Contract achieves this functionality.
AuthorizationManager.sol The AuthorizationManager contract manages access control for critical contract functions. It allows an owner to grant and revoke access to authorized addresses. **Functionality Confirmed:** - Only the contract owner can add/remove authorized addresses. - Reentrancy protection is not directly required but is enforced via access control checks. - The contract emits events when changes to authorized users occur, ensuring transparency. **Key Features:** - Owner-controlled access list. - Prevents unauthorized modifications to critical contracts.	Contract achieves this functionality.

- Ensures only approved entities can execute privileged operations.	
NumberRegistry.sol The NumberRegistry maps human-readable durations (e.g., "Month") to numerical values used in recurring payment intervals. **Functionality Confirmed:** - Ensures consistent interval definitions across all contracts. - Prevents unauthorized changes to time interval values. **Key Features:** - Converts terms like "Every Week" into machine-readable values. - Supports multilingual mappings (e.g., English and Spanish). - Ensures standardized recurrence intervals.	Contract achieves this functionality.
PaymentAgreement.sol **Functionality Confirmed:** - 1.Signature Verification: ○ Ensure EIP712 signature validation is correctly implemented. ○ Prevent replay attacks by enforcing a unique nonce per transaction. ○ Ensure that the correct signer is being verified. - 2. Access Control: ○ Confirm that only authorized users or the owner can withdraw funds. ○ Ensure that unauthorized users cannot process bulk payments. - 3. Token Transfers:	Contract achieves this functionality.

○ Ensure only supported tokens (registered in TokenRegistry) are accepted. ○ Verify that payments are executed atomically to prevent fund loss. ○ Ensure no overflow/underflow vulnerabilities in amount calculations. - 4. Nonce Management: ○ Verify that each signed transaction uses a unique nonce. ○ Ensure nonces are correctly stored and retrieved. **Key Features:** - Accepts cryptographically signed one-time payment agreements. - Allows bulk payment processing. - Supports AA Wallets (Account Abstraction) for contract-based signing. - Ensures secure and atomic execution.	
RecurringPaymentAgreement.sol **Functionality Confirmed:** - 1. Automated Execution Risks: ○ Ensure scheduled transactions execute at correct intervals. ○ Prevent unauthorized actors from triggering - 2. Signature & Nonce Security: ○ Verify that EIP712 signatures cannot be reused across payments. ○ Ensure replay protection is in place. - 3. Payment Flow Integrity: ○ Ensure that subscription cancellations immediately prevent further Transactions.	Contract achieves this functionality.

○ Confirm that overpayment and underpayment scenarios are handled safely. - 4. Smart Contract Upgradability & Governance: ○ Verify that any updates to the contract can't subvert previous agreements or alter them in any way. - 5. Storage & Gas Optimization: ○ Ensure that long-term storage of recurring payment data does not introduce excessive gas costs. ○ Confirm that batch execution of payments remains efficient. **Key Features:** - Supports automated crypto subscription payments. - Executes recurring payments securely on-chain. - Implements merchant-settlement automation. - Ensures compliance with registered tokens and addresses.	
TokenRegistry.sol The TokenRegistry contract provides a mapping of token symbols and currencies to contract addresses, ensuring that stablecoins and other tokens used in payment agreements are correctly resolved. **Functionality Confirmed:** - Only authorized entities can update the registry. - Ensures valid token addresses are mapped, reducing the risk of malicious token replacement. - Supports cross-network compatibility by resolving tokens uniquely per **Key Features:** - Maintains a list of supported tokens.	Contract achieves this functionality.

- Ensures standardized currency resolution for payments. - Prevents unauthorized modifications to the token list.	

Code Quality

This audit scope involves the smart contracts of the Coinsub project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Coinsub project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] PaymentAgreement#processBulkPayments, RecurringPayment#processBulkPayments - Redundant modifier invocation in the loop

Description

The `processBulkPayments` function internally calls the `agree` function in each iteration of the loop and both `processBulkPayments` and `agree` functions have the same modifier (`whenNotPaused`).

As a result, this modifier is invoked multiple times within the same execution of the external function, leading to redundant checks and potential inefficiencies.

The same issue for the `processBulkPayments` function in the `RecurringPaymentAgreement` contract.

Recommendation

It's recommended to make an internal function that handles payments without the modifier and calls the internal function in both `processBulkPayments` and `agree` functions.

Status

Resolved

Low

[L-01] AddressRegistry, AuthorizationManager, NumberRegistry, PaymentAgreement, RecurringPaymentAgreement, TokenRegistry - Use `Ownable2Step` versions rather than `Ownable` versions

Description

`Ownable2StepUpgradeable` and `Ownable2Step` prevents the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Recommendation

Consider using `Ownable2StepUpgradeable` and `Ownable2Step` from OpenZeppelin Contracts to enhance the security of your contract ownership management. This contract prevents the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call.

Status

Resolved

[L-02] PaymentAgreement, RecurringPaymentAgreement - Unused event declarations

Description

The contracts declare the `AuthorizedAdded` and `AuthorizedRemoved` events but they are never used in the contracts.

Recommendation

Remove unused events in the contracts.

Status

Resolved

QA

[Q-01] Contracts - Use of floating pragma

Description

Contracts use a pragma statement that does not specify a fixed compiler version but instead allows the contract to be compiled with any compatible compiler version. This can lead to various compatibility and stability issues.

Recommendation

Consider locking the pragma version whenever possible and avoid using a floating pragma in the final deployment. Consider known bugs for the compiler version that is chosen.

Status

Resolved

Centralisation

The Coinsub project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Coinsub project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. The new smart contracts have improved transaction batching, advanced recurring payment logic, and expanded multi-chain support. These smart contracts are optimized to reliably handle large transaction volumes. Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au